

Catalyst: AI-Powered Timetable Generator

Catalyst is a modern, full-stack web application for generating, visualizing, and managing academic timetables using AI (Genetic Algorithm + LLM-powered summaries). It supports admin, teacher, and student roles, real-time conflict detection, and change requests—all with a beautiful, responsive UI.

Features

- **AI Timetable Generation:** Uses a genetic algorithm to create conflict-free timetables for schools/colleges.
- **Data Upload:** Import students, teachers, rooms, and courses from CSV files.
- **Personalized Views:** Teachers and students get their own filtered schedules.
- **Conflict Detection:** Real-time highlighting of timetable conflicts.
- **Change Requests:** Teachers/students can request changes; admins can approve/reject.
- **Manual Edits:** Admins can manually adjust slots and resolve conflicts.
- **Modern UI:** Built with Next.js, Tailwind, and Radix UI for a clean, responsive experience.
- **MongoDB Backend:** All data is stored in a local MongoDB database (compatible with MongoDB Compass).
- **Genkit AI Flows:** LLM-powered summaries and reasoning for timetable explanations.

Tech Stack

- **Frontend:** Next.js 15, React 18, Tailwind CSS, Radix UI, TypeScript
- **Backend:** Next.js API routes, MongoDB (local, via Compass), Zod validation
- **AI/ML:** Custom genetic algorithm, Genkit LLM flows
- **Other:** PapaParse (CSV), jose (JWT), Socket.IO (for real-time, optional)

Folder Structure

```
src/
  app/                # Next.js app directory (pages, API routes, layouts)
    api/              # API endpoints (data, auth, timetable, requests)
    dashboard/        # Main dashboard UI (admin/teacher/student views)
    login/            # Login page
    register/         # Registration page
    components/       # UI components (dashboard, tables, dialogs, etc.)
  lib/               # Shared logic (types, schemas, utils, genetic algorithm)
  server/            # MongoDB connection and helpers
  ai/                # Genkit LLM flows and dev scripts
docs/               # Project blueprints and documentation
.env               # Environment variables (MongoDB URI, JWT secret, etc.)
```

Setup & Running Locally

1. Install dependencies:

```
npm install
```

2. Configure environment:

- Copy `.env.example` to `.env` (or edit `.env` directly):
 - `MONGODB_URI=mongodb://localhost:27017/`
 - `MONGODB_DB=catalyst`
 - `JWT_SECRET=your-very-secure-secret-here`

3. Start MongoDB:

- Make sure MongoDB is running locally (install MongoDB Community Edition or use MongoDB Compass).
- The database and collections will be created automatically on first use.

4. Run the Next.js app:

```
npm run dev  
# App runs at http://localhost:9002
```

5. (Optional) Run Genkit AI flows:

```
npm run genkit:watch
```

Usage

Admin

- Upload CSVs for students, teachers, rooms, and courses.
- Generate timetable (AI-powered) and review conflicts.
- Manually edit slots or resolve conflicts.
- Approve/reject change requests from teachers/students.

Teacher/Student

- Log in to see your personalized timetable.
- Request changes to your schedule if needed.

Architecture

- **API routes** handle CRUD for all entities and timetable generation.
 - **Genetic algorithm** (see `src/lib/genetic-algorithm.ts`) creates optimal timetables.
 - **MongoDB** stores all data (students, teachers, rooms, courses, timetables, change requests).
 - **JWT-based auth** for secure login and session management.
 - **LLM/Genkit** (optional) for advanced summaries and explanations.
-

Contributing

1. Fork this repo and clone your fork.
 2. Create a new branch for your feature/fix.
 3. Make your changes and add tests if needed.
 4. Run `npm run lint` and `npm run typecheck` to ensure code quality.
 5. Commit and push, then open a pull request.
-

License

MIT License. See [LICENSE](#) for details.